Introdução ao Modelo Físico

Introdução

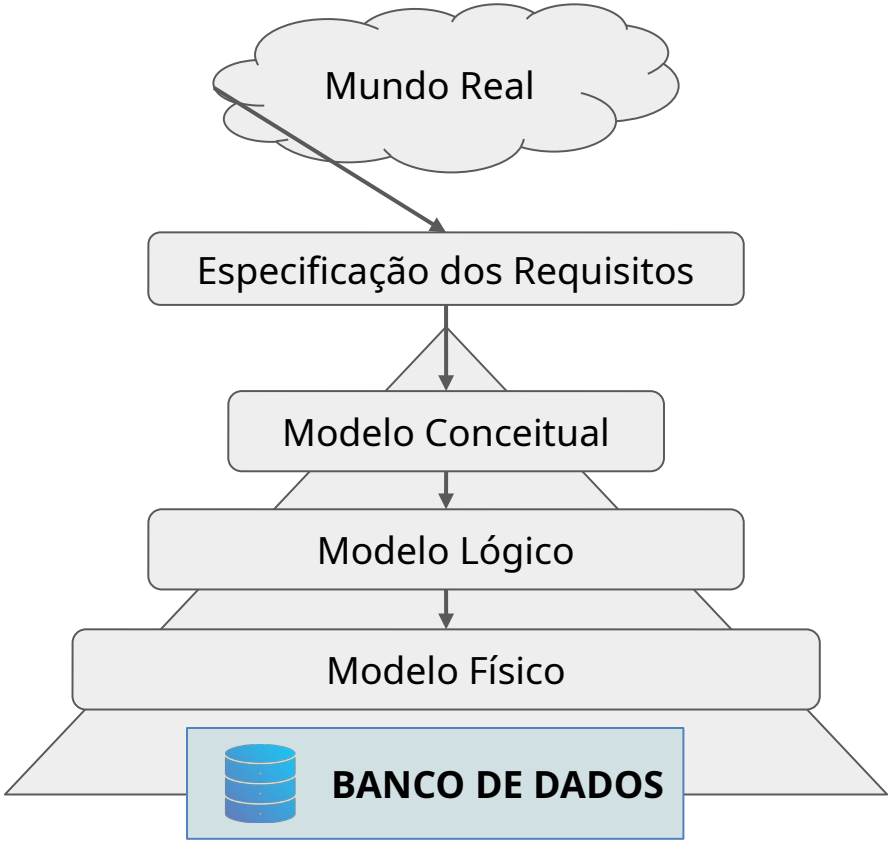
- Usa como base o modelo lógico.
- Refinamento do modelo lógico
- Preocupa em criar as estruturas de dados físicas no SGBD.
- É a etapa final antes de se partir para a criação do banco de dados.

Introdução

- Muitas das ferramentas de modelagem disponíveis no mercado possibilitam a geração do modelo físico a partir do modelo lógico desenvolvido.

- Scripts em uma linguagem denominada:
  - SQL
  - DDL

Introdução


Mundo Real
Especificação dos Requisitos
Modelo Conceitual
Modelo Lógico
Modelo Físico
BANCO DE DADOS

Introdução



- É **fonte dos requisitos** e o ponto de partida.
- Representa o ambiente sobre o qual o banco de dados é construído, incluindo os **processos de negócios** e as necessidades de informação.

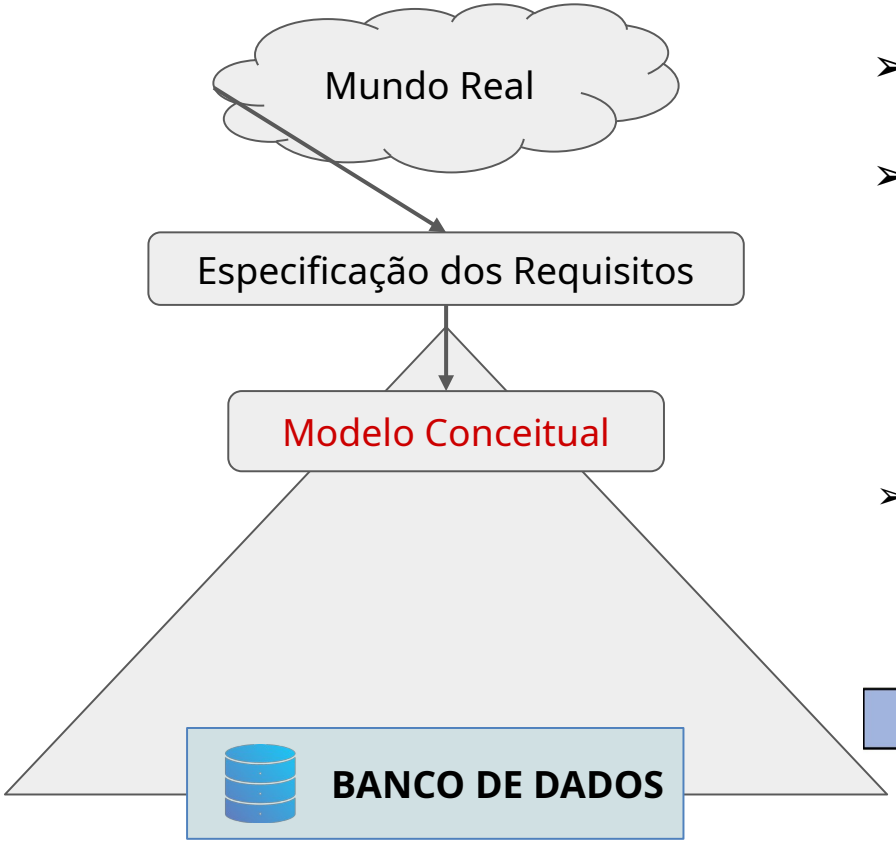
Introdução



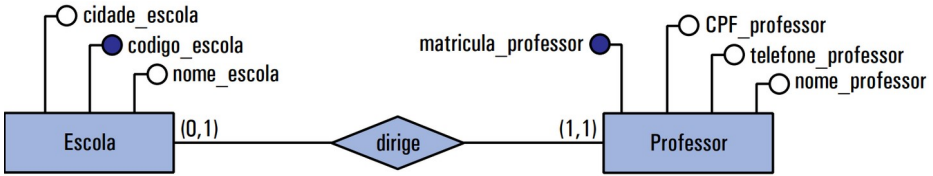
- É o processo de **coleta e análise** das necessidades de informação que o sistema de banco de dados deve atender.

- Entender o que os **usuários finais** esperam do banco de dados em termos de **armazenamento de dados**, **consultas**, **atualizações** e **relatórios**.

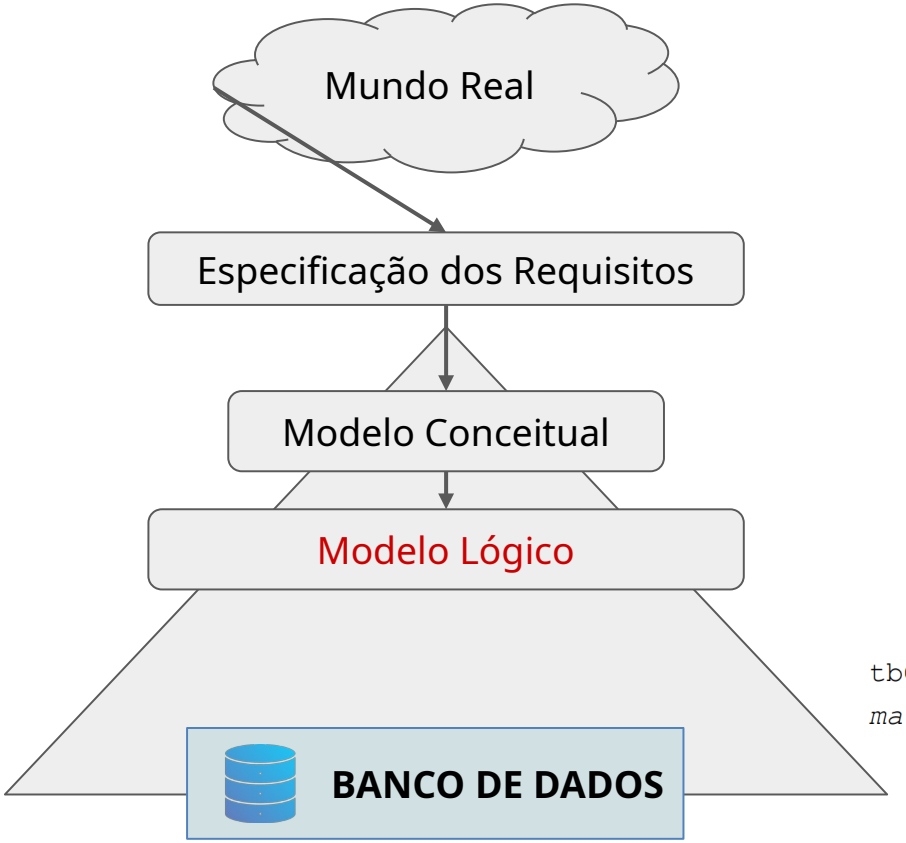
Introdução



- **Modelo abstrato**
- Organiza os dados de acordo com as regras e estruturas do mundo real, *sem levar em conta como esses dados serão armazenados fisicamente*.
- **Diagrama entidade-relacionamento (ER)**



Introdução

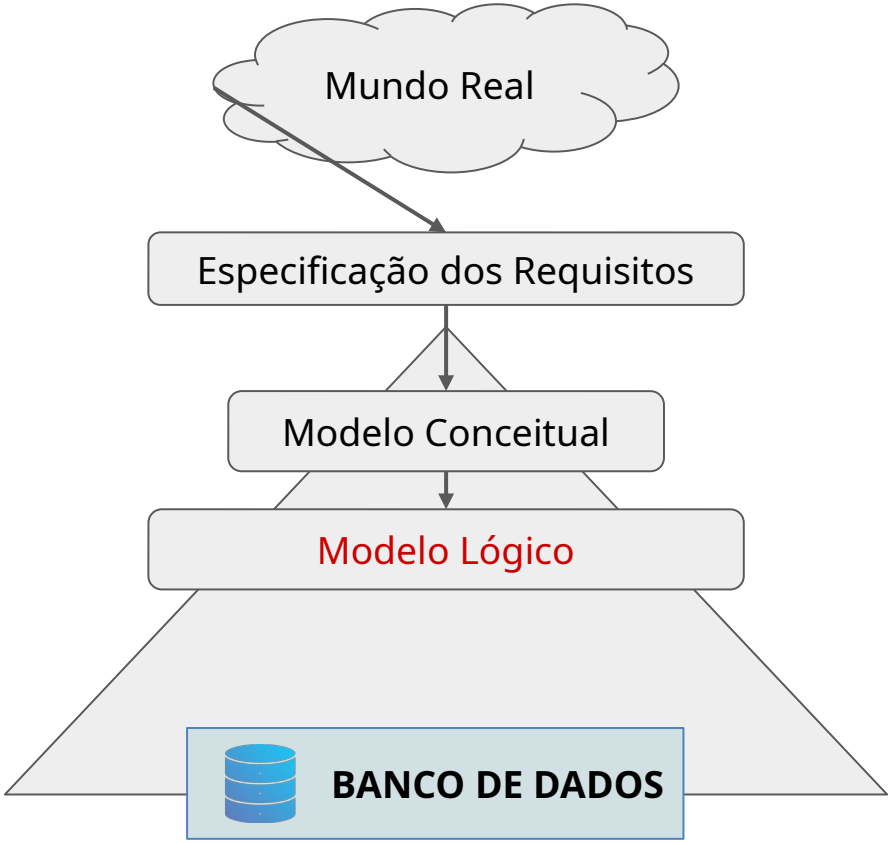


- O modelo conceitual é **traduzido** para um modelo lógico, que é **independente** do SGBD em que são implementados

- Envolve *definir* tabelas, chaves primárias e estrangeiras, e os **relacionamentos** entre as tabelas.

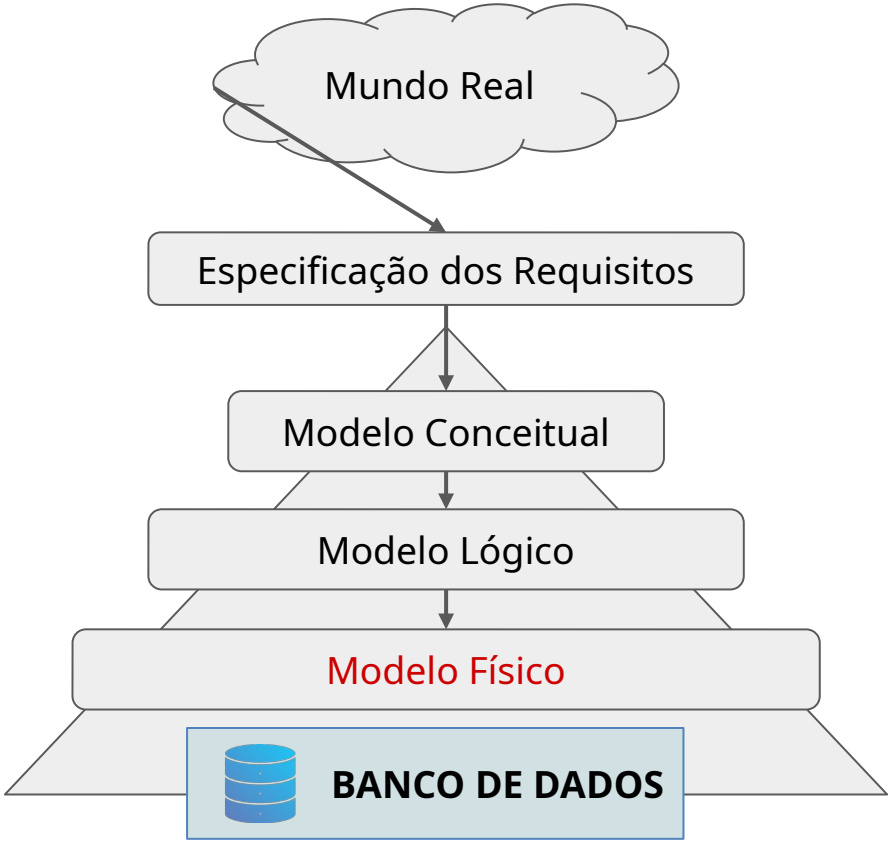
```
tbContato(matricula_aluno, fone_contato, nome_contato
matricula_aluno referencia tbAluno
```

Introdução



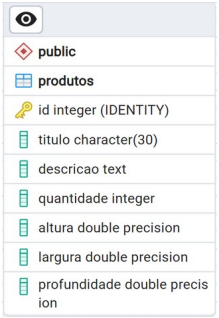
- **Normalização**
  - Evitar a redundância de dados.
  - Melhorar a integridade dos dados.
  - Facilitar a manutenção dos dados.
  - Otimizar o desempenho das consultas.
  - Flexibilidade do banco de dados.

Introdução



- O **modelo lógico** é transformado em um **modelo físico**, que é a representação de *como os dados serão realmente armazenados em um meio físico*.

- Leva em conta o SGBD específico.




```
CREATE TABLE produtos
(
    id integer NOT NULL,
    titulo character(30) NOT NULL,
    descricao text,
    quantidade integer,
    altura double precision,
    largura double precision,
    profundidade double precision,
    CONSTRAINT produtos_pkey PRIMARY KEY (id)
)
```



Structured Query Language (SQL)

SQL

- É uma linguagem para banco de dados **relacional**.
- Antes era chamada de Sequel (*Structured English Query Language*).
- Desenvolvida pela **IBM** no início da década de **70**, por Codd.
- A linguagem evoluiu bastante, passando a se chamar simplesmente Structured Query Language (SQL).

SQL

- Houve diversas variações da SQL (**dialetos**).
- A American National Standards Institute (**ANSI**) apresentou uma padronização em 1986 e a **ISO** em 1987.

- Sybase Advantage Database Server
- Apache Derby
- Caché
- DB2
- Dataflex
- Firebird
- HSQLDB
- IDMS
- IMSInformix
- Ingres
- InterBase
- Microsoft Access
- Microsoft SQL Server
- MySQL
- Oracle
- PointBase Micro
- PostgreSQL
- SQLite
- LiteBase Mobile
- Sybase Adaptive Server Enterprise
- Teradata

https://www.iso.org/standard/76583.html
https://blogs.oracle.com/sql/post/general-availability-of-the-sql2023-standard

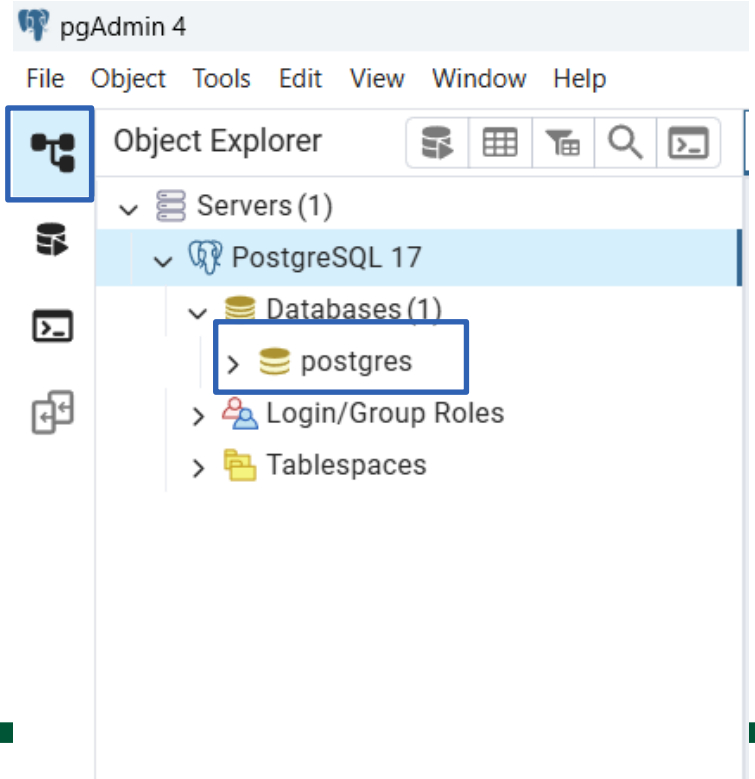
Estrutura da SQL

- A SQL é composta por um conjunto de linguagens que permitem a **definição**, **consulta** e **atualização de dados**, além de recursos adicionais para a *segurança e gestão* do BD.
  - **DDL:** Linguagem de Definição de Dados;
  - **DQL:** Linguagem de Consulta de Dados
  - **DML**: Linguagem de Manipulação de Dados;
  - **DCL**: Linguagem de Controle de Dados;
  - **DTL**: Linguagem de Controle de Transações.

Mãos à Obra!



- Abram o **pgAdmin**

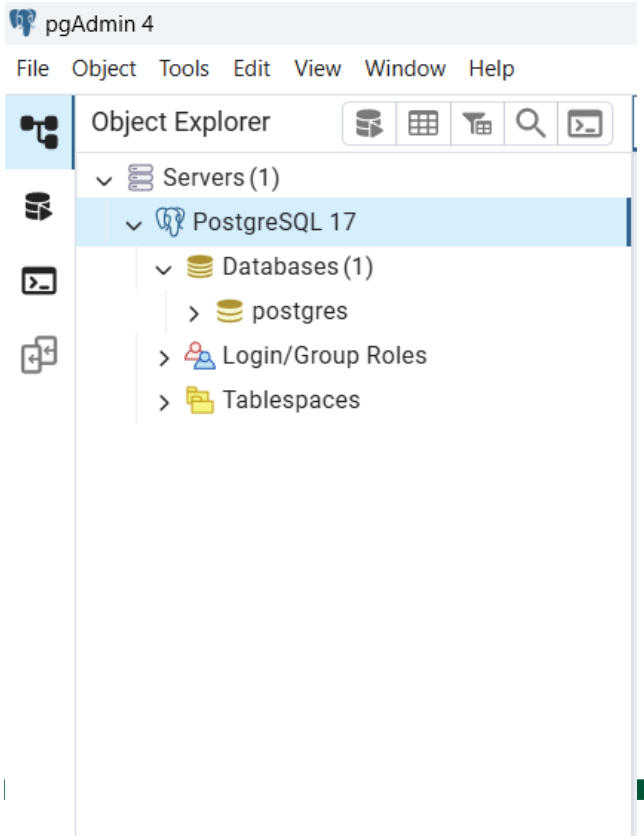


- Quando você instala o PostgreSQL, o sistema automaticamente cria alguns DB

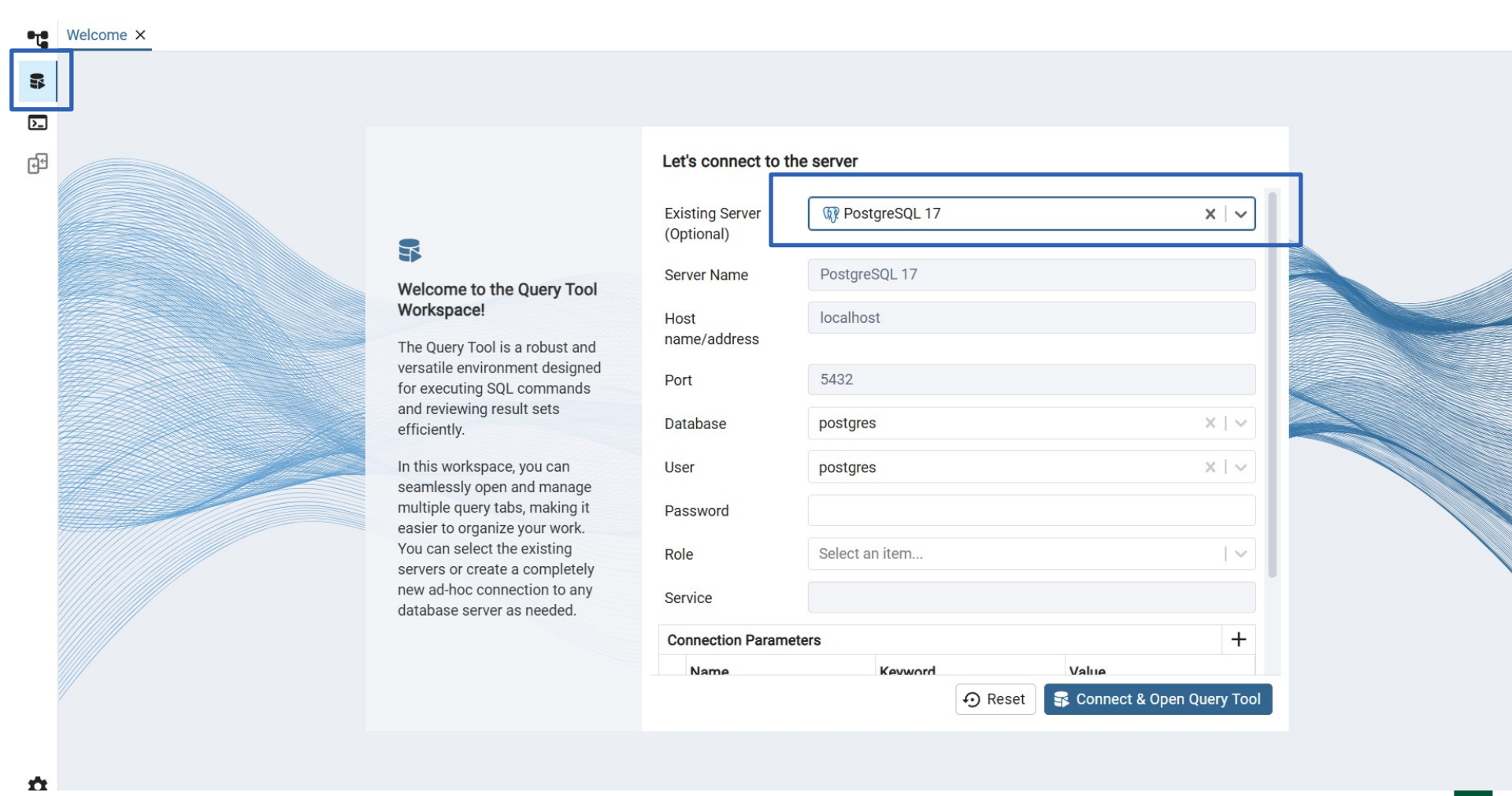
- DB **postgres**
  - Esse banco é criado para que o administrador possa ter um ponto de acesso inicial ao servidor e executar comandos administrativos ou criar novos bancos.

Mãos à Obra!





- Um banco de dados chamado **template1:**
  - Serve como modelo para criação de novos bancos. Sempre que você cria um banco novo sem especificar um modelo, o PostgreSQL copia a estrutura e os objetos do template1.
- Um banco chamado **template0:**
  - Usado como uma versão "limpa" do banco, sem personalizações, para permitir restaurações ou criação de bancos sem herdar modificações feitas no template1.


Welcome
Welcome to the Query Tool Workspace!
The Query Tool is a robust and versatile environment designed for executing SQL commands and reviewing result sets efficiently.
In this workspace, you can seamlessly open and manage multiple query tabs, making it easier to organize your work. You can select the existing servers or create a completely new ad-hoc connection to any database server as needed.
Let's connect to the server
Existing Server (Optional)
PostgreSQL 17
Server Name
PostgreSQL 17
Host name/address
localhost
Port
5432
Database
postgres
User
postgres
Password
Role
Select an item...
Service
Connection Parameters
Name
Keyword
Value
Reset
Connect & Open Query Tool

Linguagem de Definição de Dados (DDL)

- **Criar um Database (Banco de Dados):**

Sintaxe

```
CREATE DATABASE nome_do_banco_de_dados;
```

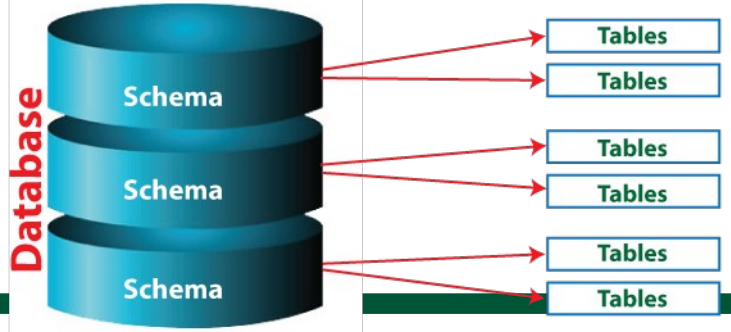
Data Output | Messages | Notifications

```
CREATE DATABASE

Query returned successfully in 277 msec.
```

```
CREATE DATABASE loja_do_joao;
```

- **Criar um Schema (Esquema):**
  - Após criar o banco de dados e conectar-se a ele, você pode criar um schema
  - Um **schema** no PostgreSQL funciona como um contêiner lógico dentro de um banco de dados, atuando como um namespace para organizar objetos do banco, como tabelas, views, funções, tipos de dados, procedures armazenadas e outros objetos.
  - De forma muito simplificada, o schema é como uma “pasta” dentro de um banco de dados, que organiza tabelas. views. funções.....

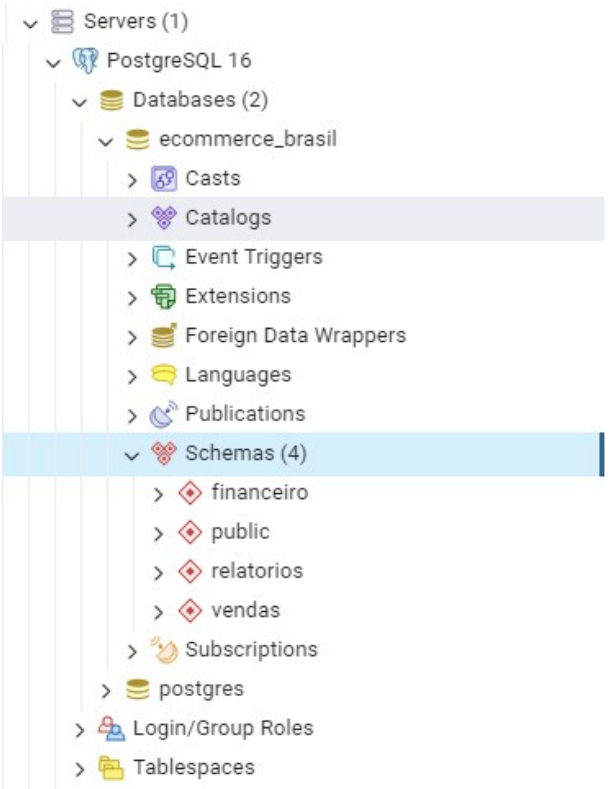


Linguagem de Definição de Dados (DDL)

- **Criar um Schema (Esquema):**

Sintaxe

```
CREATE SCHEMA nome_do_esquema;
```



Linguagem de Definição de Dados (DDL)

- **Criar uma Table (Tabela):**
  - Dentro de um schema, você pode criar tabelas para definir a estrutura onde os dados serão armazenados:

Sintaxe

```
CREATE TABLE nome_do_esquema.nome_da_tabela (
    coluna1 tipo_de_dado1 CONSTRAINTS,
    coluna2 tipo_de_dado2 CONSTRAINTS,
    coluna3 tipo_de_dado3 CONSTRAINTS,
    ...
);
```

Linguagem de Definição de Dados (DDL)

Sintaxe

```
CREATE TABLE nome_do_esquema.nome_da_tabela (
    coluna1 tipo_de_dado1 CONSTRAINTS,
    coluna2 tipo_de_dado2 CONSTRAINTS,
    coluna3 tipo_de_dado3 CONSTRAINTS,
    ...
);
```

- **nome_do_esquema:** O esquema ao qual a tabela pertencerá. Se você não especificar um esquema, a tabela será criada no esquema padrão (geralmente é o **public**).
- **nome_da_tabela:** O nome que você deseja dar à sua tabela.

Linguagem de Definição de Dados (DDL)

Sintaxe

```
CREATE TABLE nome_do_esquema.nome_da_tabela (
    coluna1 tipo_de_dado1 CONSTRAINTS,
    coluna2 tipo_de_dado2 CONSTRAINTS,
    coluna3 tipo_de_dado3 CONSTRAINTS,
    ...
);
```

- **coluna:** O nome de uma coluna na tabela.
- **tipo_de_dado:** O tipo de dados (como **INTEGER**, **VARCHAR**, **TIMESTAMP**) que a coluna irá armazenar.

Linguagem de Definição de Dados (DDL)

Sintaxe

```
CREATE TABLE nome_do_esquema.nome_da_tabela (
    coluna1 tipo_de_dado1 CONSTRAINTS,
    coluna2 tipo_de_dado2 CONSTRAINTS,
    coluna3 tipo_de_dado3 CONSTRAINTS,
    ...
);
```

- **CONSTRAINTS:** Restrições opcionais para a coluna, como **NOT NULL, PRIMARY KEY, UNIQUE, etc**.

- **Criar uma Table (Tabela):**
  - Dentro de um schema, você pode criar tabelas para definir a estrutura onde os dados serão armazenados:

```sql
CREATE TABLE loja.cliente (
    id_cliente SERIAL PRIMARY KEY,
    nome VARCHAR(100) NOT NULL,
    cpf CHAR(11) UNIQUE NOT NULL,
    data_nascimento DATE,
    data_de_criacao TIMESTAMP DEFAULT CURRENT_TIMESTAMP
);
```